

HELEP L3 Patterns-in-Code Document

Part A: Pre-Implemented and Completed Patterns

A.1 Choreographed Saga

The SOS workflow is implemented as a choreographed saga. No central service directly commands every step. Instead, each service reacts to Kafka events and emits the next event in the workflow.

The saga begins when `sos-service` handles `POST /sos`, stores the incident, and publishes `sos.triggered` with the incident ID as the key (`services/sos-service/app/main.py:78`, `services/sos-service/app/main.py:84`, `services/sos-service/app/main.py:85`).

`dispatch-service` consumes `sos.triggered`, checks idempotency, selects a responder, reserves the responder, and publishes `responder.assigned` (`services/dispatch-service/app/main.py:52`, `services/dispatch-service/app/main.py:62`, `services/dispatch-service/app/main.py:77`).

`notification-service` consumes `responder.assigned`, `safety.zone.entered`, and offline `sos.triggered` events, persists simulated notifications, logs delivery, and emits `notification.sent` (`services/notification-service/app/main.py:52`, `services/notification-service/app/main.py:63`, `services/notification-service/app/main.py:67`).

The compensation step is SOS cancellation. `sos-service` publishes `sos.cancelled` (`services/sos-service/app/main.py:102`, `services/sos-service/app/main.py:109`). `dispatch-service` reacts by releasing the assignment and publishing `responder.confirmed` with `status=RELEASED` (`services/dispatch-service/app/main.py:96`, `services/dispatch-service/app/main.py:100`, `services/dispatch-service/app/main.py:101`).

A.2 Pub/Sub via Apache Kafka

Kafka is used as the event broker for service communication. Producers use `AIOKafkaProducer` (`services/user-service/app/events.py:26`) and consumers use `AIOKafkaConsumer` (`services/user-service/app/events.py:105`).

The producer enables idempotence and waits for acknowledgements from all in-sync replicas (`services/user-service/app/events.py:29`, `services/user-service/app/events.py:31`, `services/user-service/app/events.py:32`). The consumer disables auto-commit and commits manually only after the handler succeeds (`services/user-service/app/events.py:109`, `services/user-service/app/events.py:119`, `services/user-service/app/events.py:120`). This provides at-least-once processing.

Partition keying matters because saga-critical events pass `key=iid`, where `iid` is the incident ID (`services/sos-service/app/main.py:95`, `services/dispatch-service/app/main.py:80`, `services/dispatch-service/app/main.py:101`). Events with the same key remain ordered within the same partition.

A.3 Repository

Database access is isolated in `app/db.py` modules. For example, `dispatch-service` wraps SQLite connections in a repository-style context manager (`services/dispatch-service/app/db.py:12`) and exposes functions such as `all_free_responders`, `reserve_responder_for`, and `release_assignment` (`services/dispatch-service/app/db.py:66`, `services/dispatch-service/app/db.py:71`, `services/dispatch-service/app/db.py:97`).

If route handlers queried SQLite directly, business logic, SQL, transaction handling, and HTTP/event concerns would be mixed together. The Repository pattern keeps persistence behavior easier to test and reason about.

A.4 Strategy

Responder matching uses the Strategy pattern. `dispatch-service/app/matching.py` defines a matcher protocol and multiple interchangeable implementations (`services/dispatch-service/app/matching.py:27`, `services/dispatch-service/app/matching.py:31`, `services/dispatch-service/app/matching.py:42`).

The active strategy is selected through the `MATCHER` environment variable (`services/dispatch-service/app/matching.py:72`, `services/dispatch-service/app/matching.py:73`). The dispatch flow only calls `matcher().pick(...)`, so it does not need to know which algorithm is active (`services/dispatch-service/app/main.py:71`).

A third strategy, `RoundRobinMatcher`, was added (`services/dispatch-service/app/matching.py:57`). It rotates across available responders and is selected with `MATCHER=round_robin` (`services/dispatch-service/app/matching.py:76`, `services/dispatch-service/app/matching.py:77`).

A.5 Outbox-Lite

The SOS trigger flow is an outbox-lite implementation. `sos-service` first writes the incident to its database, then publishes `sos.triggered` in the same async request flow (`services/sos-service/app/main.py:84`, `services/sos-service/app/main.py:85`).

It is “lite” because it does not use a durable outbox table and a separate relay process. A production outbox would store pending events transactionally with the database write, then a background publisher would reliably publish and mark them sent.

A.6 Circuit Breaker

The Kafka publishing helper includes a circuit breaker around producer sends. The circuit breaker tracks failures, opens after the threshold, and allows retry after a reset timeout (`services/user-service/app/events.py:60`, `services/user-service/app/events.py:67`, `services/user-service/app/events.py:78`).

`publish()` checks the breaker before attempting to send (`services/user-service/app/events.py:87`, `services/user-service/app/events.py:89`). On success it resets the breaker

(`services/user-service/app/events.py:94`). On failure it records the failure and re-raises the exception (`services/user-service/app/events.py:95`, `services/user-service/app/events.py:96`).

The transition model is:

- **CLOSED**: `fails` is below the threshold and publishing is allowed.
- **OPEN**: `fails` reaches the threshold and publishing is blocked.
- **HALF_OPEN**: after `reset_after_s`, one publish attempt is allowed.
- Success resets failures and closes the circuit.
- Failure increments failures and keeps or returns the circuit to open behavior.

Part B: Patterns Added

B.1 CQRS Read Model

`analytics-service` is a CQRS-style read model. Write-side services emit domain events, while analytics consumes those events and builds query-focused statistics (`services/analytics-service/app/main.py:25`, `services/analytics-service/app/main.py:40`, `services/analytics-service/app/main.py:85`).

Problem solved: police/statistics queries do not need to read from the operational databases of SOS, dispatch, or notification services.

Trade-off: the analytics view is eventually consistent. It updates after Kafka events are consumed, not necessarily at the exact same instant the write-side event occurs.

B.2 Bulkhead via Consumer Groups

Each service uses a separate Kafka consumer group, such as `dispatch-service`, `notification-service`, and `analytics-service` (`services/dispatch-service/app/main.py:38`, `services/notification-service/app/main.py:26`, `services/analytics-service/app/main.py:23`).

Problem solved: slow analytics processing does not block notification or dispatch processing. Each service owns its consumption pace and failure boundary.

Trade-off: every service must manage idempotency and event handling independently. This increases operational complexity but improves isolation.

Part C: Anti-Patterns Avoided

The architecture avoids the shared database anti-pattern. Each service owns its own database path, configured through `DB_PATH`, and encapsulates persistence inside its own `app/db.py` module (`services/dispatch-service/app/db.py:8`, `services/sos-service/app/main.py:21`, `services/notification-service/app/main.py:18`).

The architecture also avoids synchronous fan-out from `sos-service`. SOS creation publishes `sos.triggered` and lets dispatch, notification, and analyt-

ics consume asynchronously (`services/sos-service/app/main.py:85`). This protects the user-facing SOS request from waiting on every downstream service.